

COURS DE COMPILATION

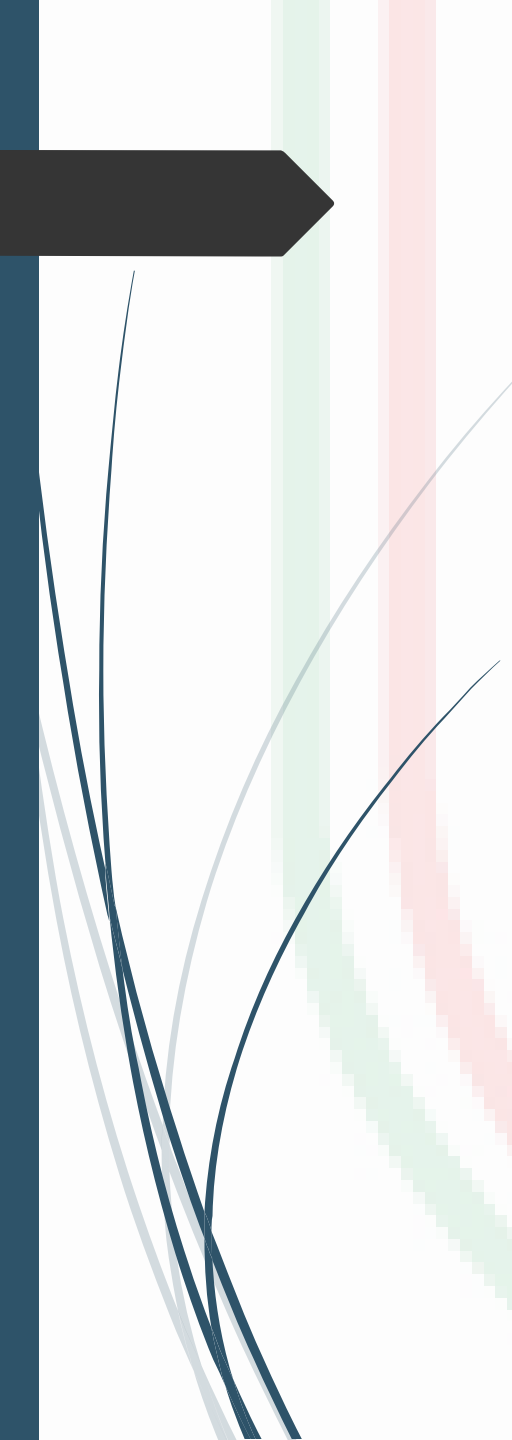
Université du Burundi
Faculté des Sciences de l'Ingénieur
Département des TIC
BAC III GI

Prof Dr. Ir. Michele MUKESHIMANA

Tel.77067444/67928044

email: michele.mukeshimana@ub.edu.bi

Juillet 2025



PLAN DU COURS

Chap1 : Introduction

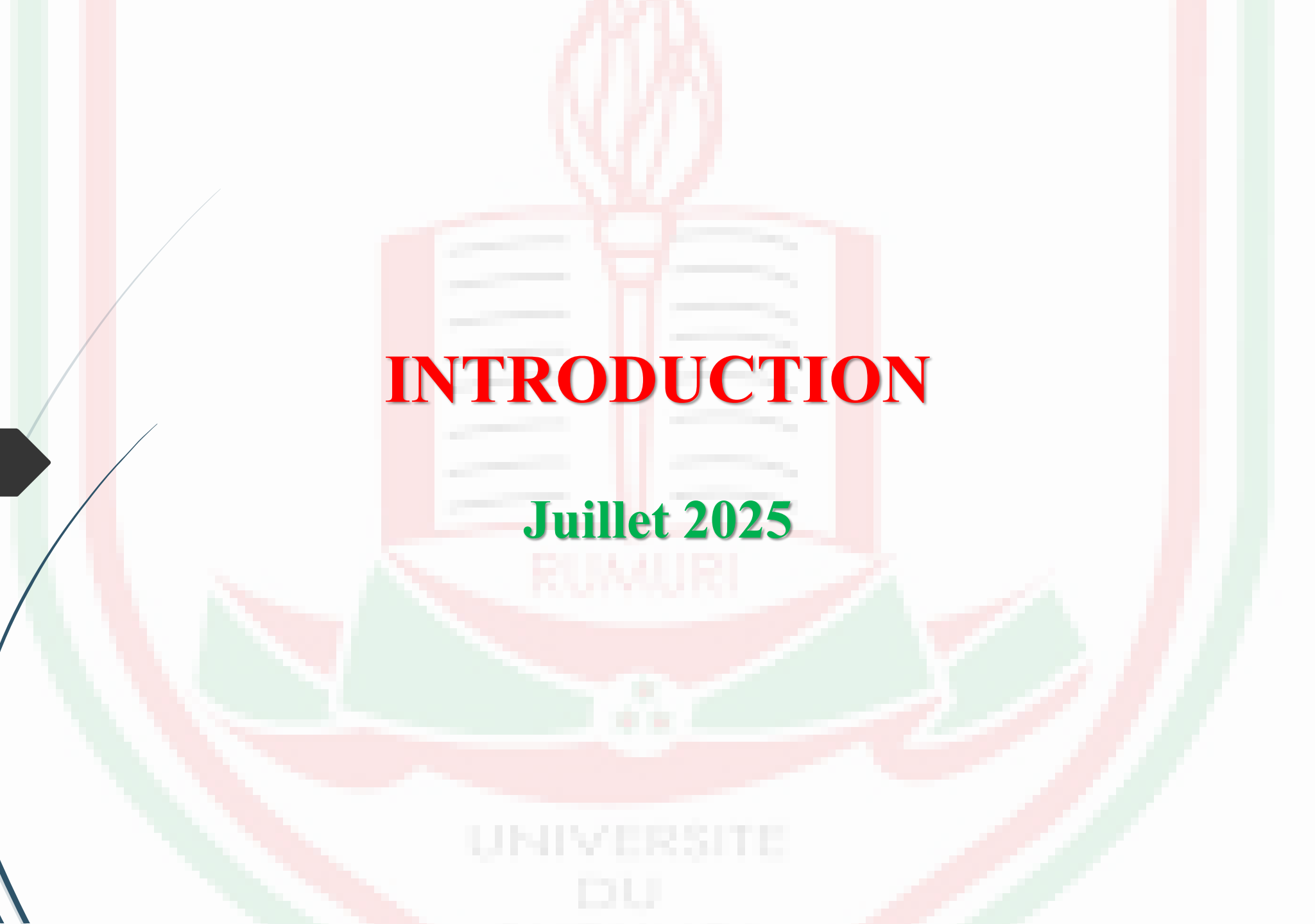
Chap2 : Analyse lexicale

Chap3 : Analyse syntaxique

Chap4 : Analyse sémantique

Chap5 : Production de code

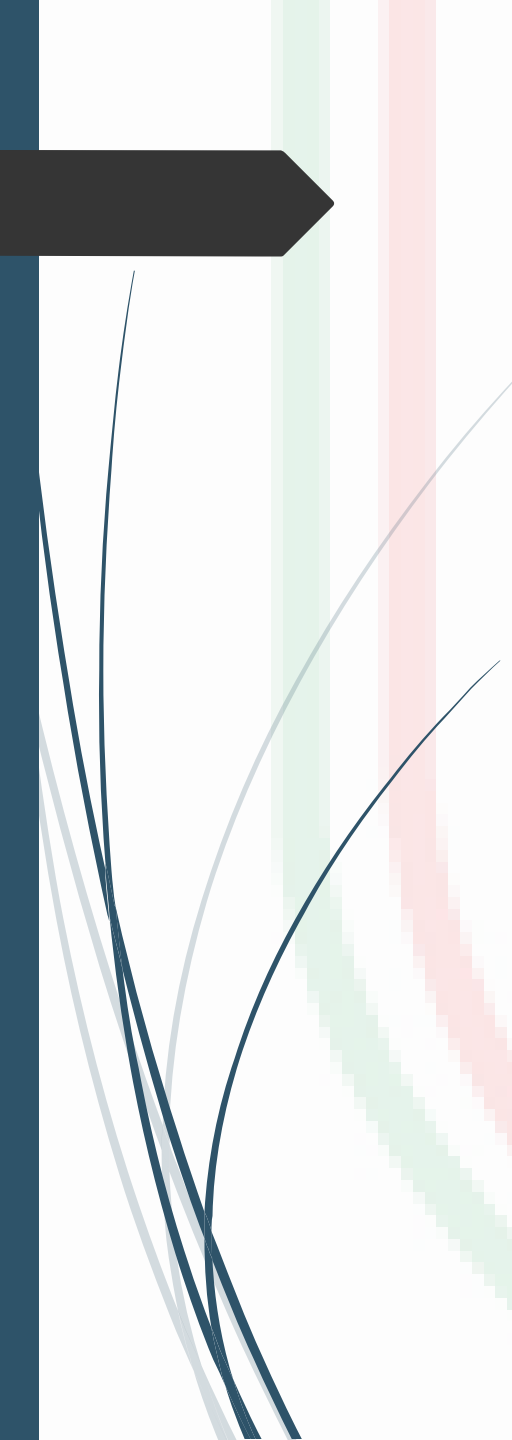
UNIVERSITE
DU

The background features a large, faint watermark of the University of Rumuri logo. The logo is a shield-shaped emblem. At the top is a torch with a flame. Below the torch is an open book with text on its pages. The word 'RUMURI' is written in a stylized font across the middle of the shield. The bottom of the shield is divided into green and red sections. The entire emblem is encircled by a green and red border. On the left side of the slide, there is a dark blue vertical bar with a black arrow pointing right towards the center.

INTRODUCTION

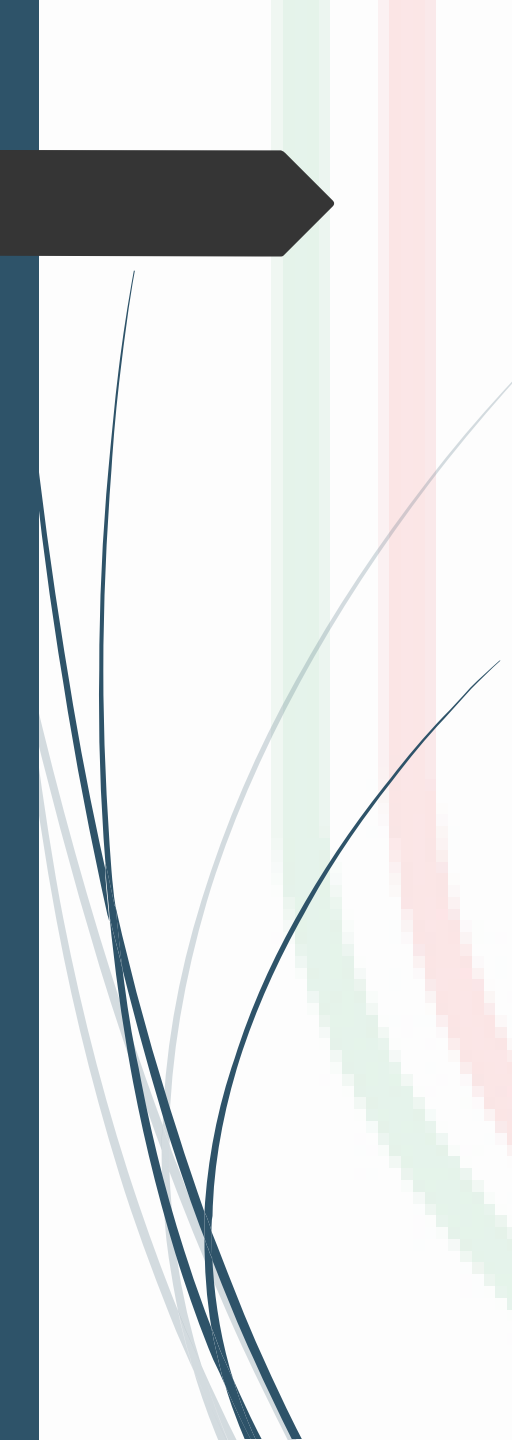
Juillet 2025

UNIVERSITE
DU



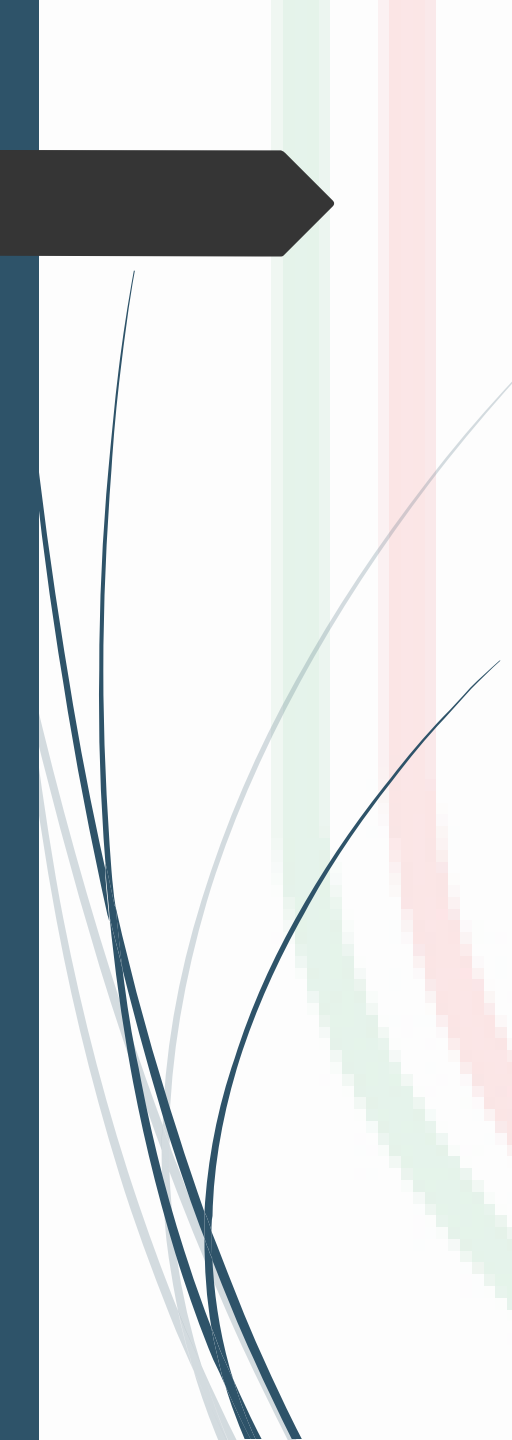
Introduction

- **Tous les ingénieurs informaticiens ne participent pas au développement de compilateurs.**
- **Par contre, la plupart écrivent des programmes, et donc utilisent des compilateurs ou des interpreteurs.**



Introduction

- ➡ Pour bien maîtriser les langages de programmation, il est important de comprendre d'une part les principes de conception des langages de programmation, et d'autre part les transformations de programmes qui sont réalisées par les compilateurs.



Introduction

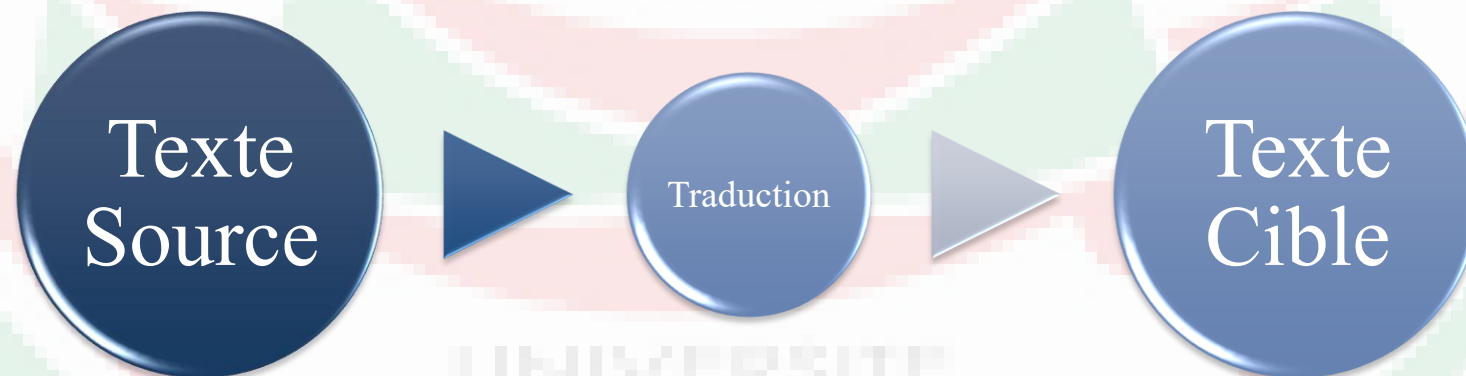
- ➡ **Ce cours permet d'obtenir des connaissances de base sur la théorie des langages de programmations et leur traduction en langage compréhensible par l'ordinateur.**

Introduction

- **Un programme informatique :**
 - **Une pensée exprimée en un texte**
- **Ce texte désigne une suite d'opérations à effectuer en vue d'obtenir un résultat.**
- **Ce texte est conçu dans un langage**
- **Ce langage a besoin d'être compréhensible sinon une traduction est nécessaire**

Introduction

- La machine (Ordinateur) ne comprend qu'un langage dit langage machine ou bien langage binaire. Il est dit binaire car il est composé par une suite de bits 0 et 1.
- Tout programme en un autre langage est traduit en langage machine.



Historique des Langages de Programmation

Langage machine

- 0A43

Langage symbolique (assembleur)

- MOV A0,A1

Langage de haut niveau

- Affranchis de la machine

Compilateur et interpréteur

- Le programme en langage de haut niveau a besoin d'être traduit en langage compréhensible par la machine :

Compilation

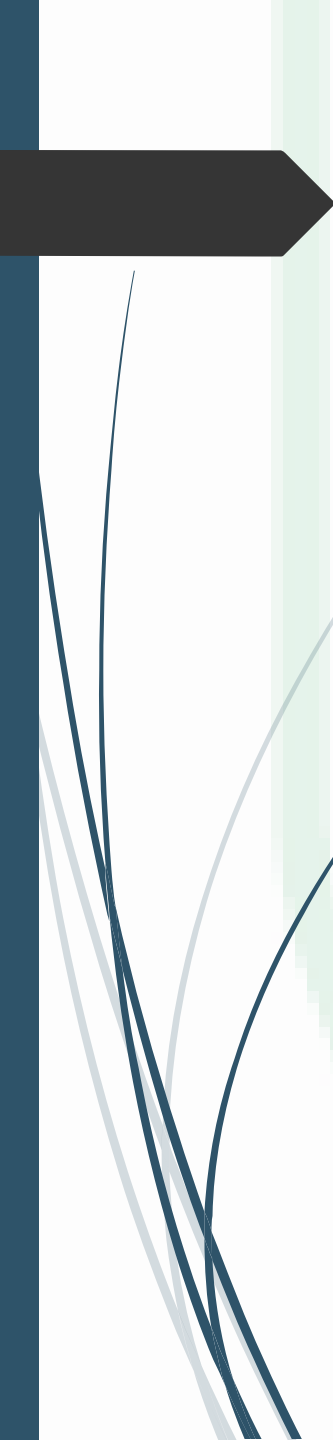
- Un compilateur :
 - du langage de haut niveau en langage cible
 - Traduction une fois pour toutes
- Un interpréteur :
 - Du langage de haut niveau en langage cible
 - Traduction progressive

Compilateur et interpréteur

- Un programme traduit par un compilateur est plus lente à traduire qu'un programme traduit par un interpréteur
- Un programme compilé est plus rapide en exécution qu'un programme interprété
- Un compilateur ne constate des erreurs statique sans exécution alors qu'interpréteur constate erreur de façon dynamique , lors de l'exécution du programme.

Pourquoi apprendre la compilation

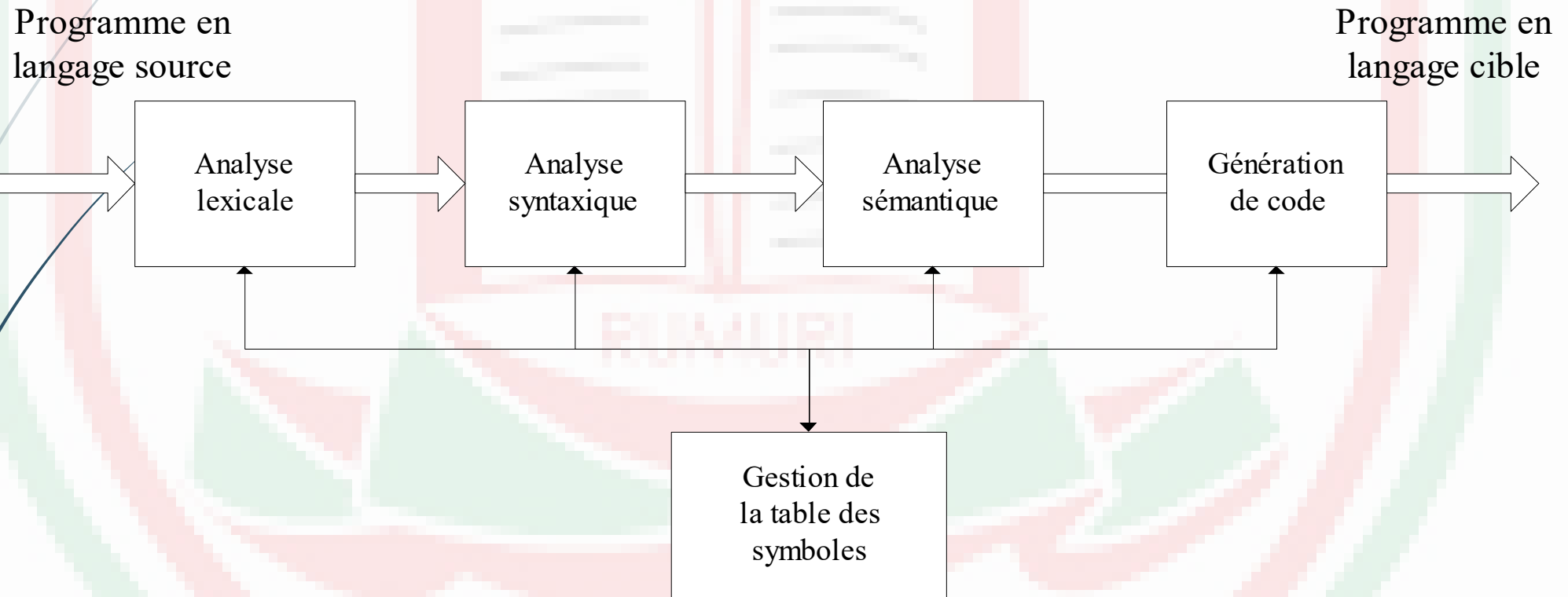
- ➡ L'étude de la compilation nous permet de voir en détail le fonctionnement d'un langage de programmation, y compris dans des aspects obscurs et peu étudiés.
- ➡ Pour écrire du bon code, il faut avoir une idée de la façon dont il est traduit. L'étude de la compilation donc permet d'avoir une orientation dans l'écriture des langages de programmation et leur exploitation.



Caractéristique d'un bon compilateur

- ➡ Reconnaître tout le langage, et rien que le langage
- ➡ Ne pas se restreindre à des constructions humainement lisibles
- ➡ Indiquer de façon claire les erreurs et éviter les messages d'erreur en cascade
- ➡ Traduire correctement vers le langage cible, et en donnant le meilleur code possible
- ➡ Être raisonnablement rapide

Phases de la compilation



Analyse lexicale

- Décomposer la suite de caractères (syllabes) en mots constitutifs, ou **TOKENS**.
- Exemple: for i :=1 to vmax do a :=a+i;

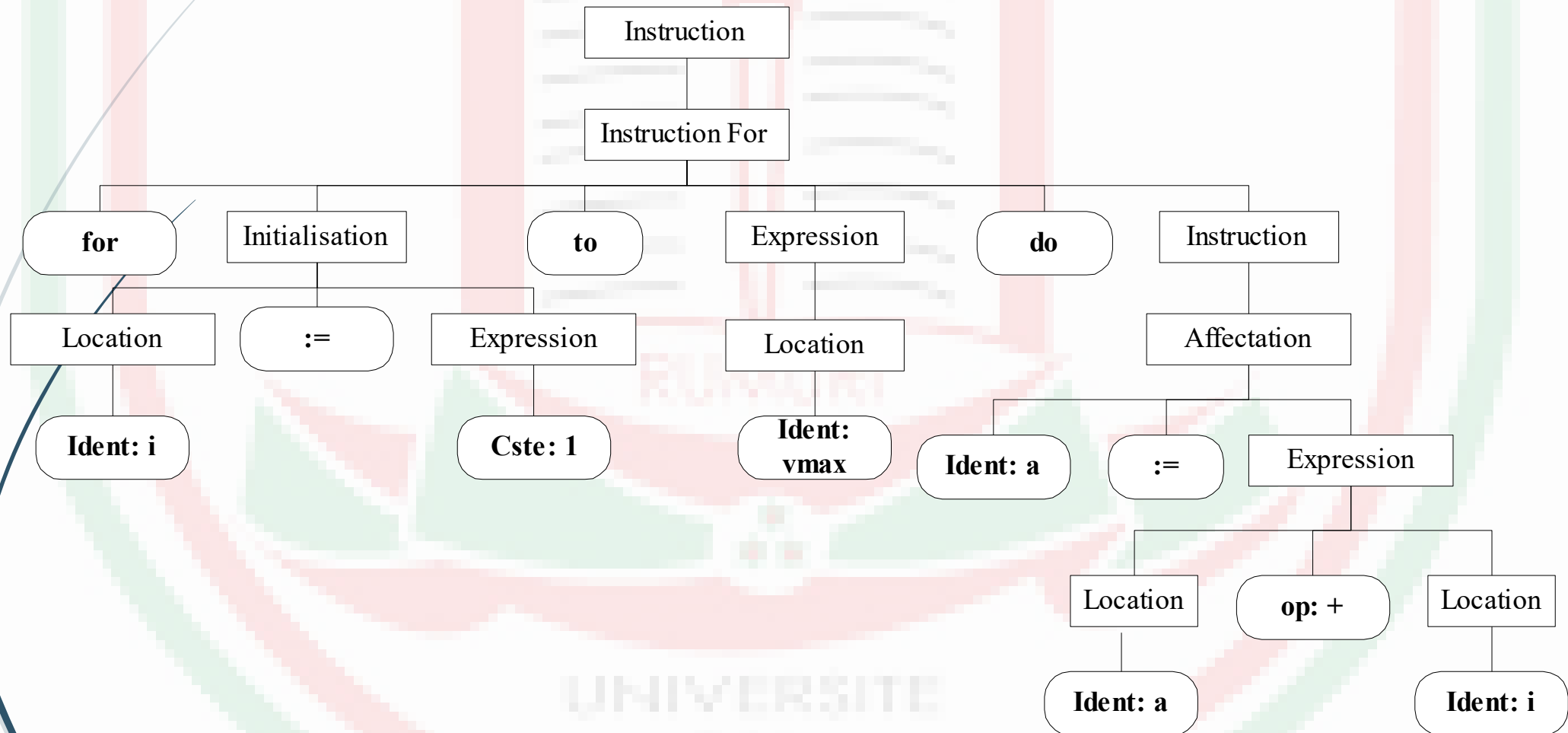
```
for  : mot clé
i    : identificateur
:=   : affectation
1    : entier
to   : mot clé
vmax : identificateur
do   : mot clé
a    : identificateur
:=   : affectation
a    : identificateur
+    : opérateur arithmétique
i    : identificateur
;    : séparateur
```

Analyse syntaxique

- Vérification grammaticale
- L'analyseur syntaxique regroupe les mots produits par l'analyseur lexical en phrases, en identifiant **le rôle de chacun des mots**.
- Les phrases en sortie sont souvent représentées sous la forme d'un arbre syntaxique

Exemple: `for i :=1 to vmax do a :=a+i;`

Analyse syntaxique

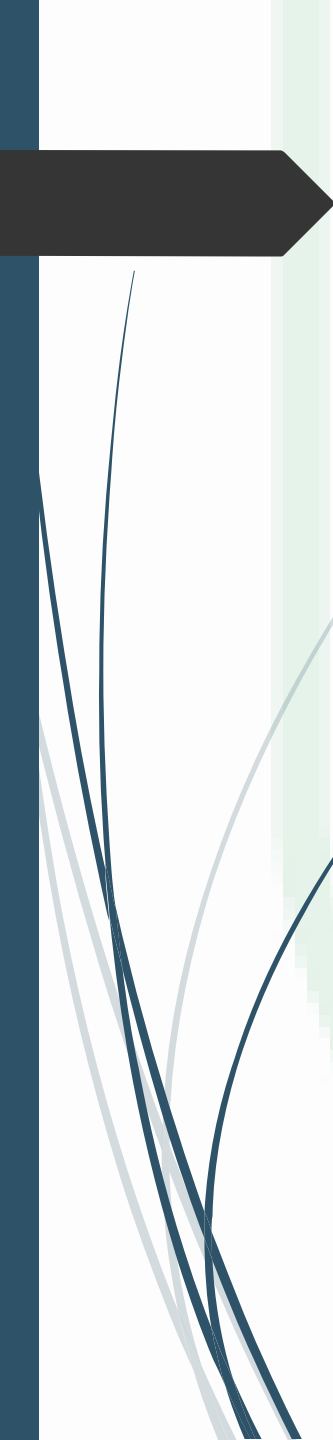


Analyse sémantique

- Vérification de la correspondance entre **variable et leur type**

Exemple : dans l'exemple précédent il faut vérifier si l'identificateur 'i' est de type 'entier' et la variable 'a' est un nombre.

- Cette opération s'effectue en parcourant l'arbre syntaxique et en vérifiant à chaque niveau que les opérations sont correctes



Generation de code

➡ Cette phase aboutit à la génération du code pour le langage machine cible.



EXERCICES

- 1. Quelle est la difference entre compilateur et interpréteur?**
- 2. Quels sont les langages de programmation compilés par des compilateurs et ceux compilés par des interpréteurs?**

Bibliography

- **Techniques et outils pour la compilation**, Henri Garreta, Faculté des Sciences de Luminy - Université de la Méditerranée, *Janvier 2001*
- **Langages et compilation**, Michaël Krajecki, Département de Mathématiques et Informatique Université de Reims Champagne-Ardenne, ,année 2003-2004.
- **Langages et Compilation**, H. Fouchal, Université de Reims Champagne-Ardenne, January, 2010
- **Compilation : théorie, techniques et outils**, H. ABDULRAB (INSA Rouen), C. MOULIN (UTC), S. TOUATI (UV SQY)
- **Compilers : Principles, Techniques, & Tools, Second Edition**, Alfred V. Aho of Columbia University, Monica S. Lam (Stanford University), Ravi Sethi (Avaya), Jeffrey D. Ullman(Stanford University), 2007